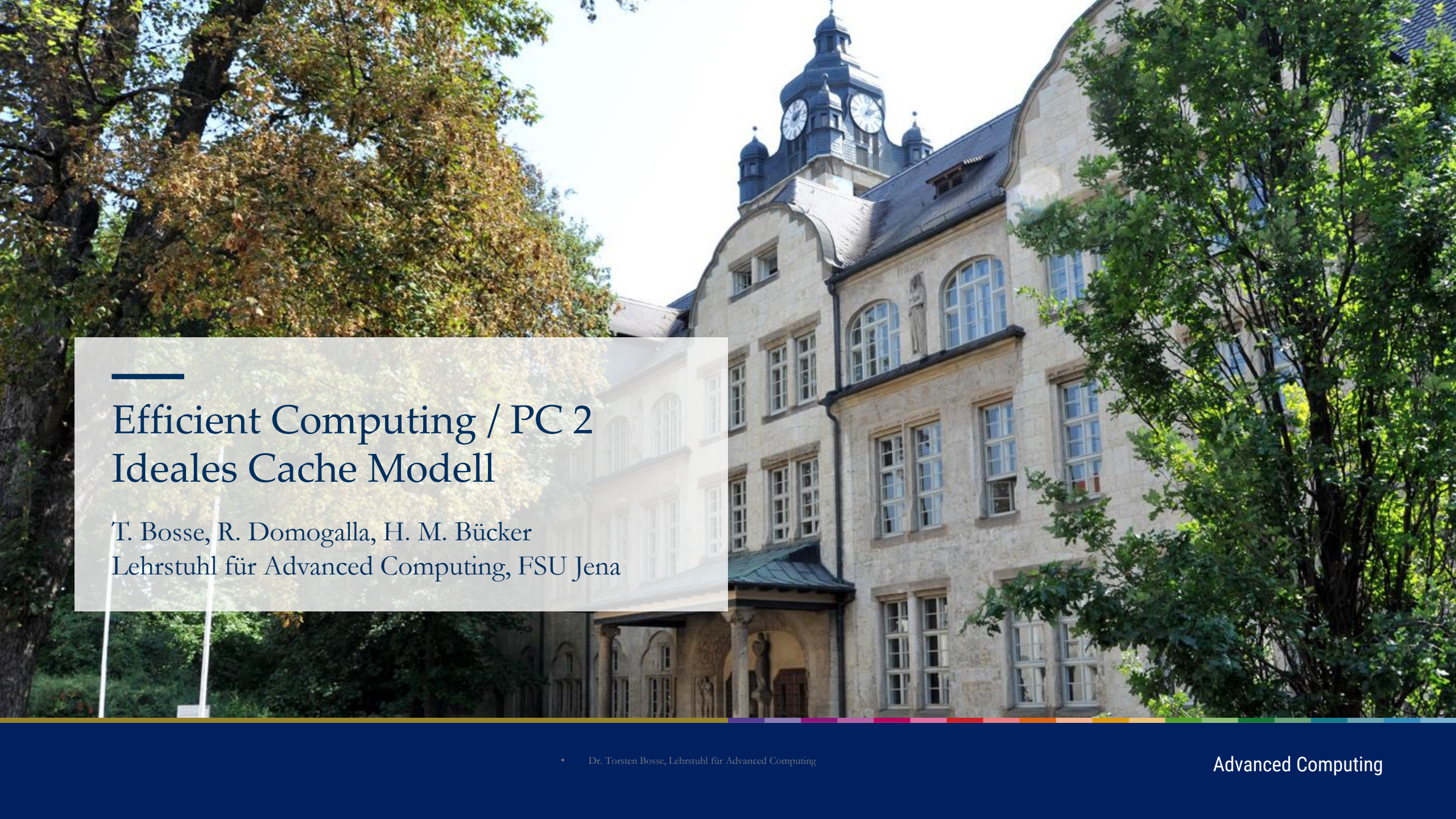




Efficient Computing / PC 2

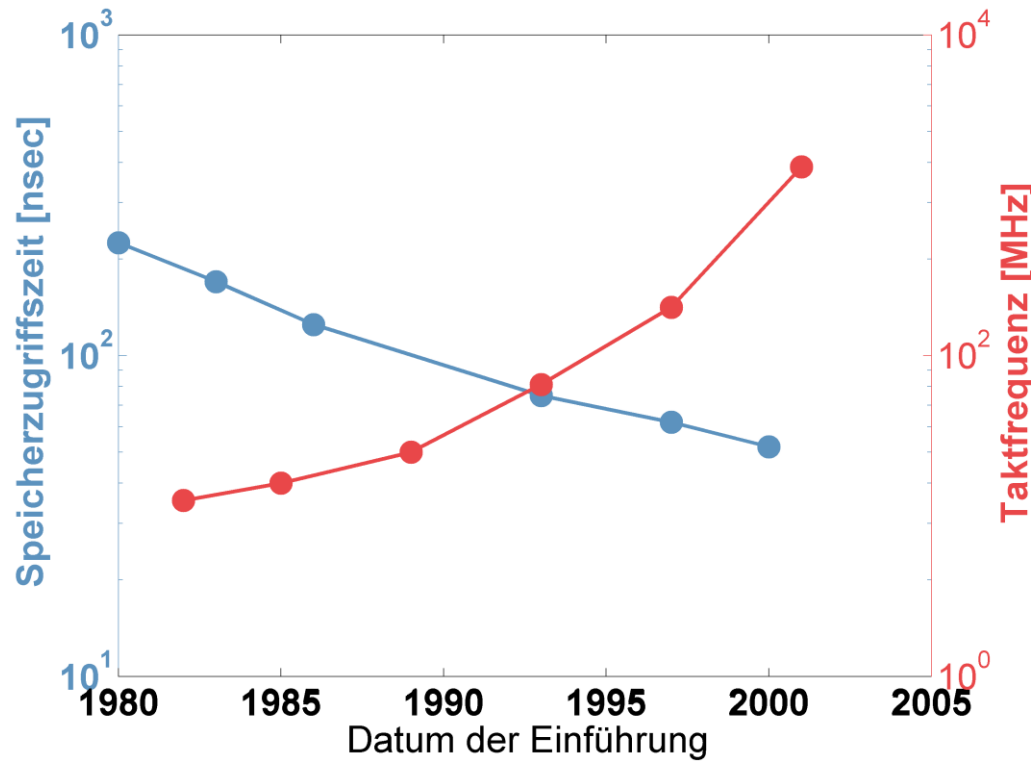
Ideales Cache Modell

T. Bosse, R. Domogalla, H. M. Bucker
Lehrstuhl für Advanced Computing, FSU Jena

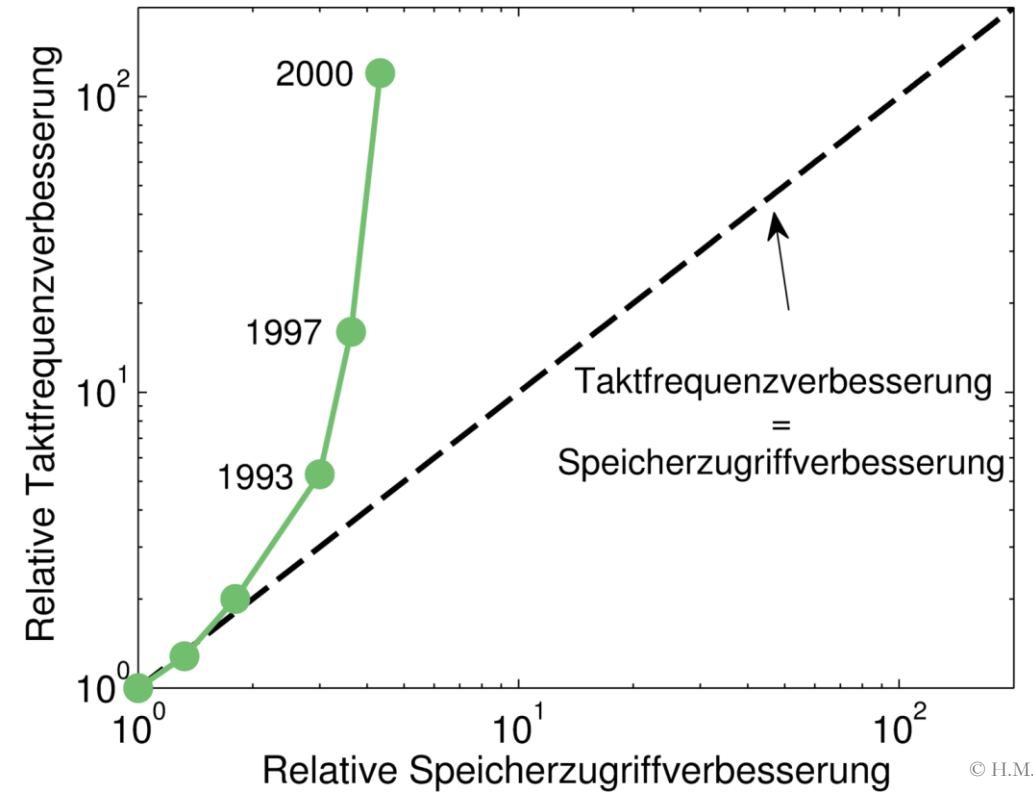
Wiederholung/Motivation

- **Bit Darstellung:** Abstraktes Konzept, verschiedene physikalische Realisierungen
 - **Miniaturisierung:** Bit Darstellung durch Transistoren erlaubt Verkleinerung.
 - **Speed-Up:** Miniaturisierung führt zu schnelleren und kleineren CPUs und Speicher.
 - **Limitations:** Miniaturisierung nur bis zu einem gewissen Grad möglich.
-
- **ABER:** Geschwindigkeit der Programme skalierte nicht entsprechend der von CPUs.
 - **Frage:** Was geht schief?

Memory-Processor Gap



© H.M. Bucker



© H.M. Bucker

RANDOM ACCESS MODEL

Problem: $z \leftarrow z + x \cdot y$, mit Skalaren z, x, y

- Move z from Memory to CPU
- Move x from Memory to CPU
- Move y from Memory to CPU
- Multiply x and y
- Add z to the result
- Move z from CPU to Memory

F Gesamtanzahl von CPU Operationen
 G Gesamtanzahl von Speicherbewegung
 $\rightarrow$ **Laufzeit:** $T = \alpha F + \beta G$

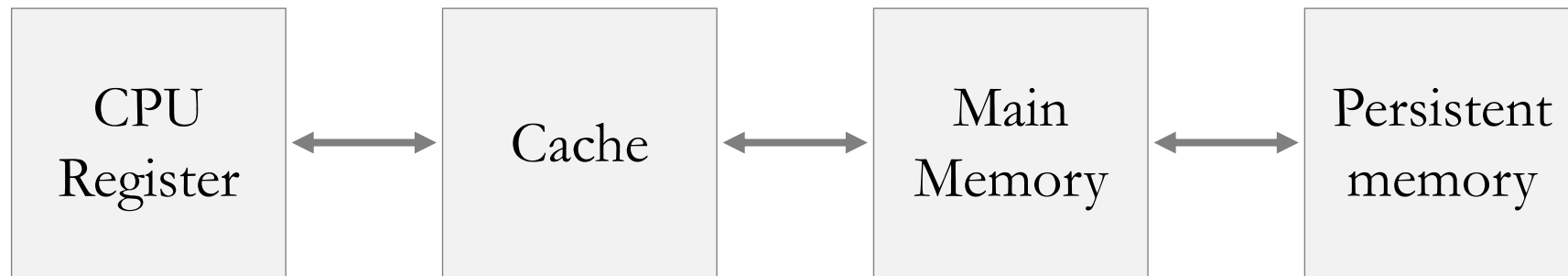
α Zeit für eine CPU Berechnung

β Zeit ein Word zu verschieben

Das vereinfachte Model $T = \alpha F$, um die Laufzeit eines Algorithmus zu approximieren, ist nur für $\alpha \gg \beta$ hinreichend, d. h., wenn die Kosten für Speicherbewegungen verhältnismässig klein sind. Dies gilt nicht für typische Werte, wie $\alpha \sim 1ns$ und $\beta \sim 50ns$ für die $\alpha \ll \beta$ gilt.

Caching

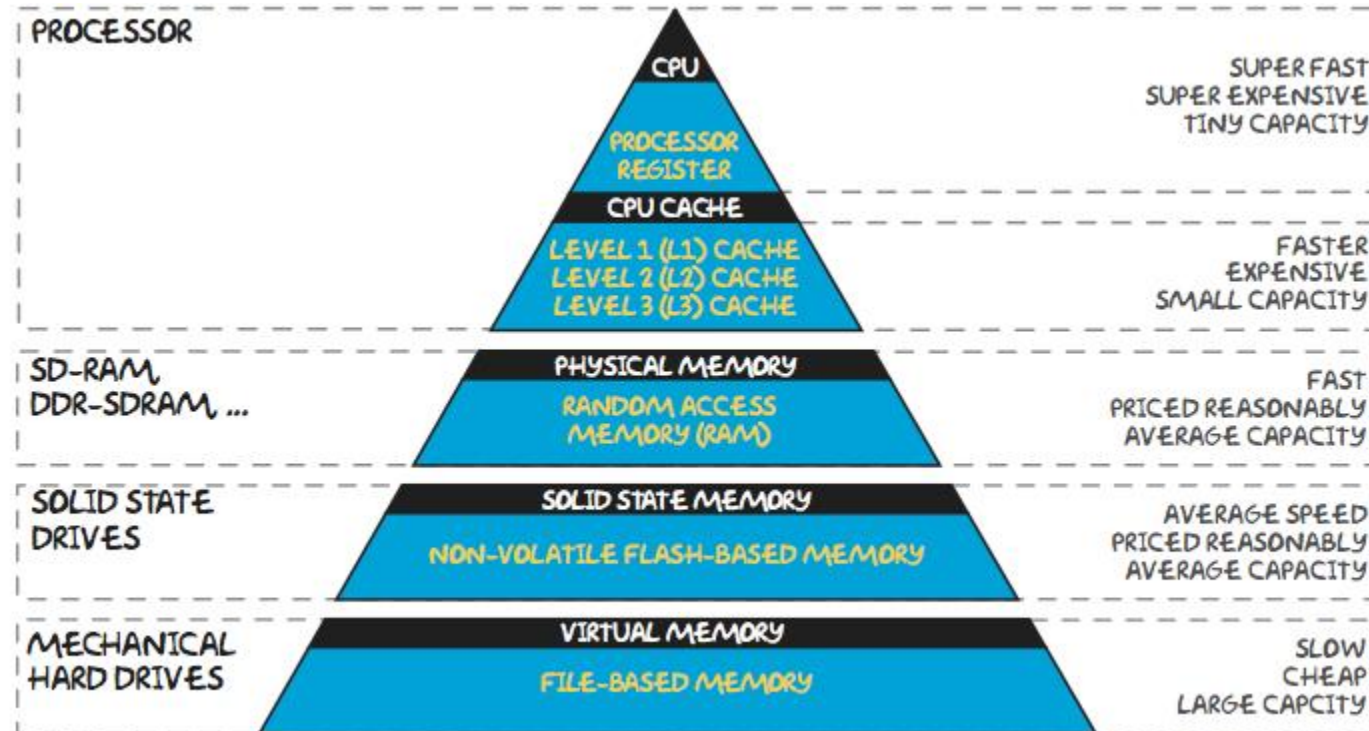
- **Idee:** Benutze eine Speicherhierarchie mit mehreren Schichten unterschiedlicher Größe.



- **Register:** Schnellster und kleinster Speicher, direkt von der CPU zugänglich.
- **Cache:** Brücke zwischen Register und Hauptspeicher; schneller als RAM, aber kleiner.
- **Hauptspeicher:** Größer, aber langsamer als Cache; Enthält aktive Daten.
- **Persistenter Speicher:** Nichtflüchtig (z. B. SSD/HDD); speichert Daten ohne Strom.

Speicherhierarchie

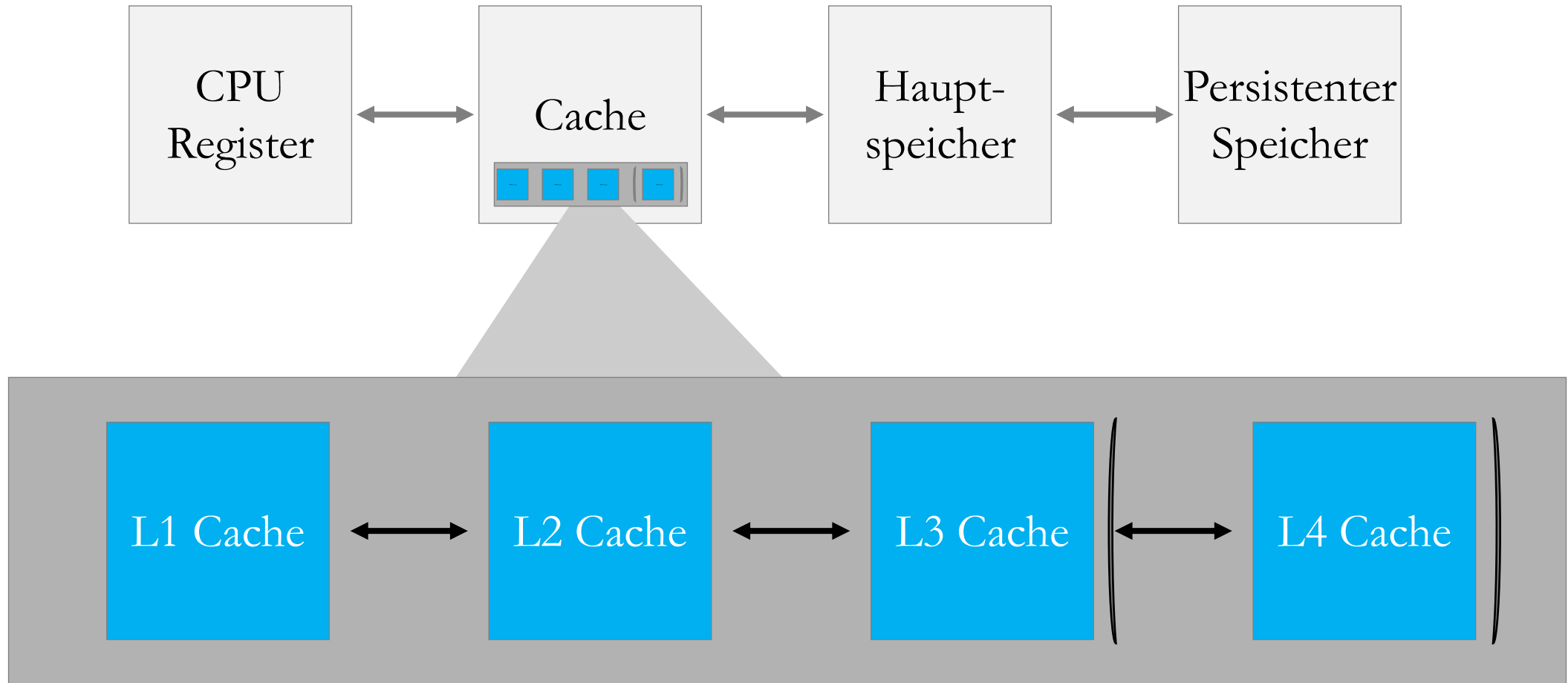
Source: Harris and Harris, Digital Design and Computer Architecture ARM Edition.



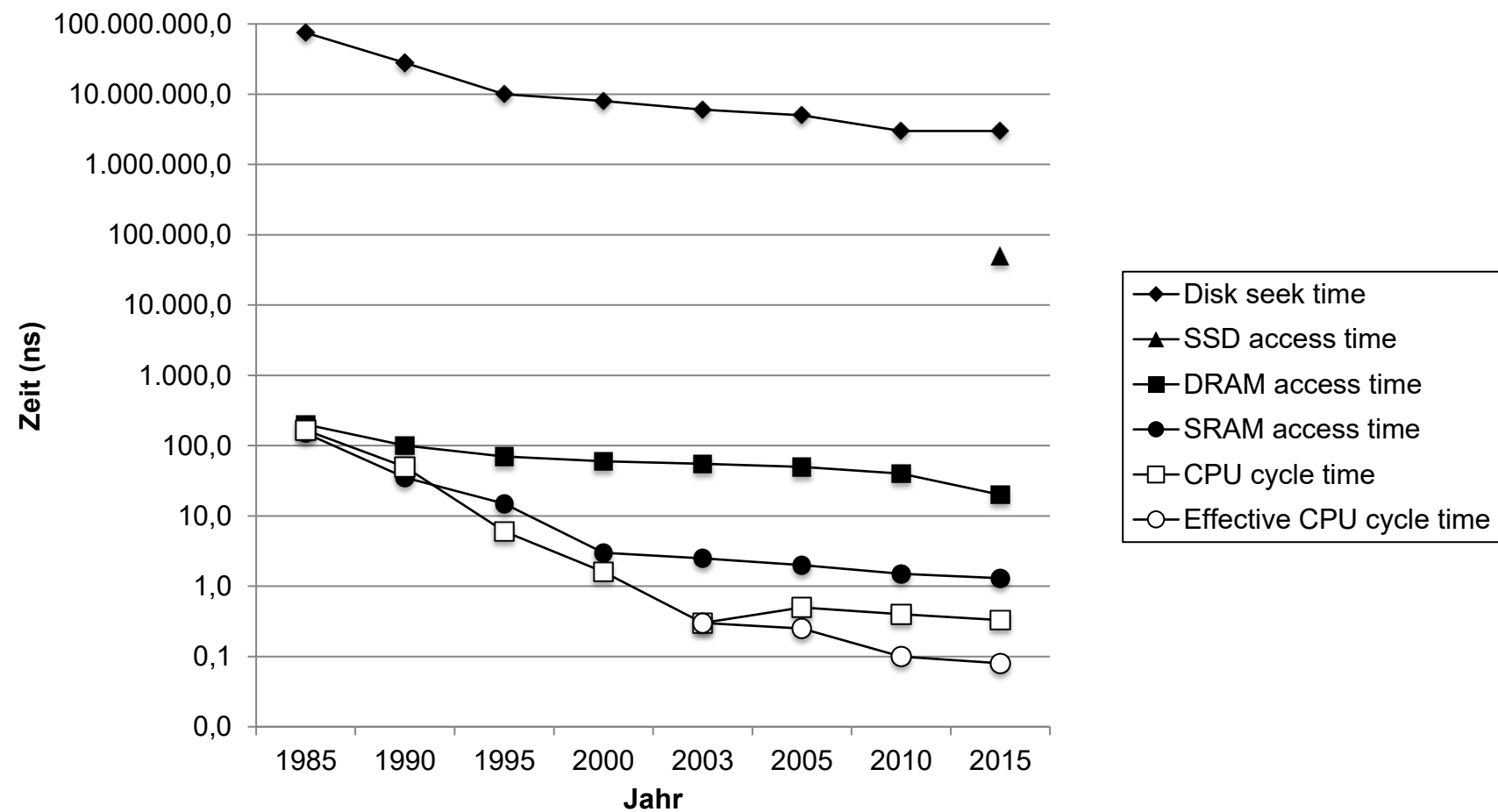
Source: https://runestone.academy/ns/books/published/welcome2/computer-architecture_memory-hierarchy.html

Technologie	Preis/GB	Zugriffszeit (ns)	Bandbreite (GB/s)
SRAM	\$5.000	0.5	25 +
DRAM	\$7	50 – 150	10
SSD	\$0.05	25.000 – 100.000	0.5

Arten von Cache



Historische Entwicklung der Speichergeschwindigkeit



Speicherkosten vs. Leistung

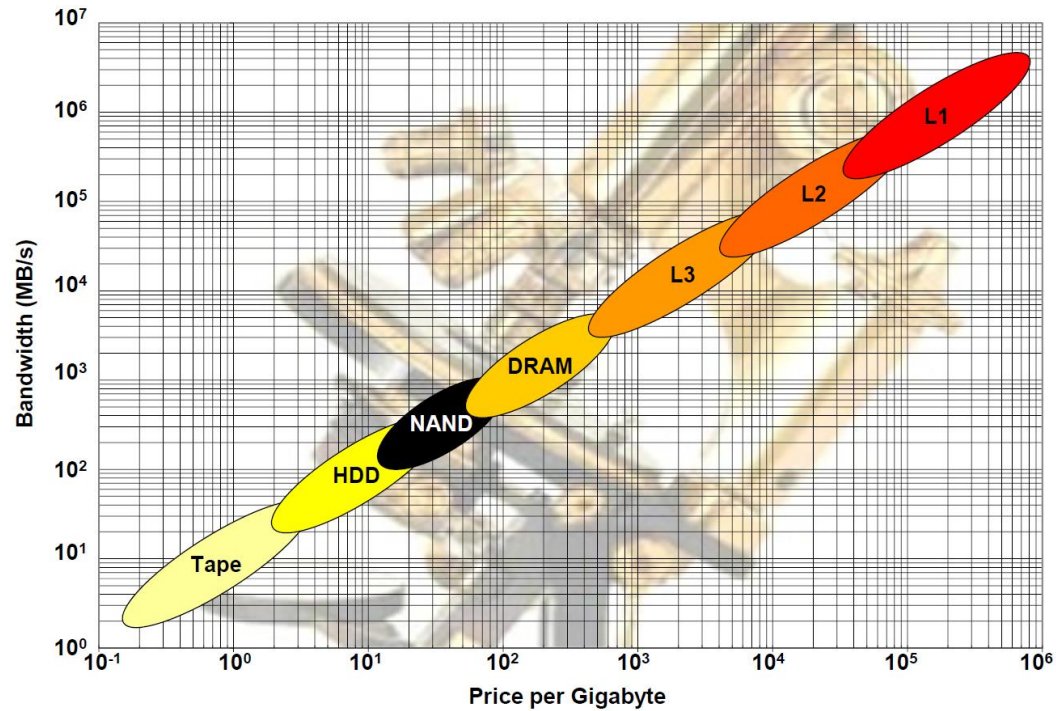
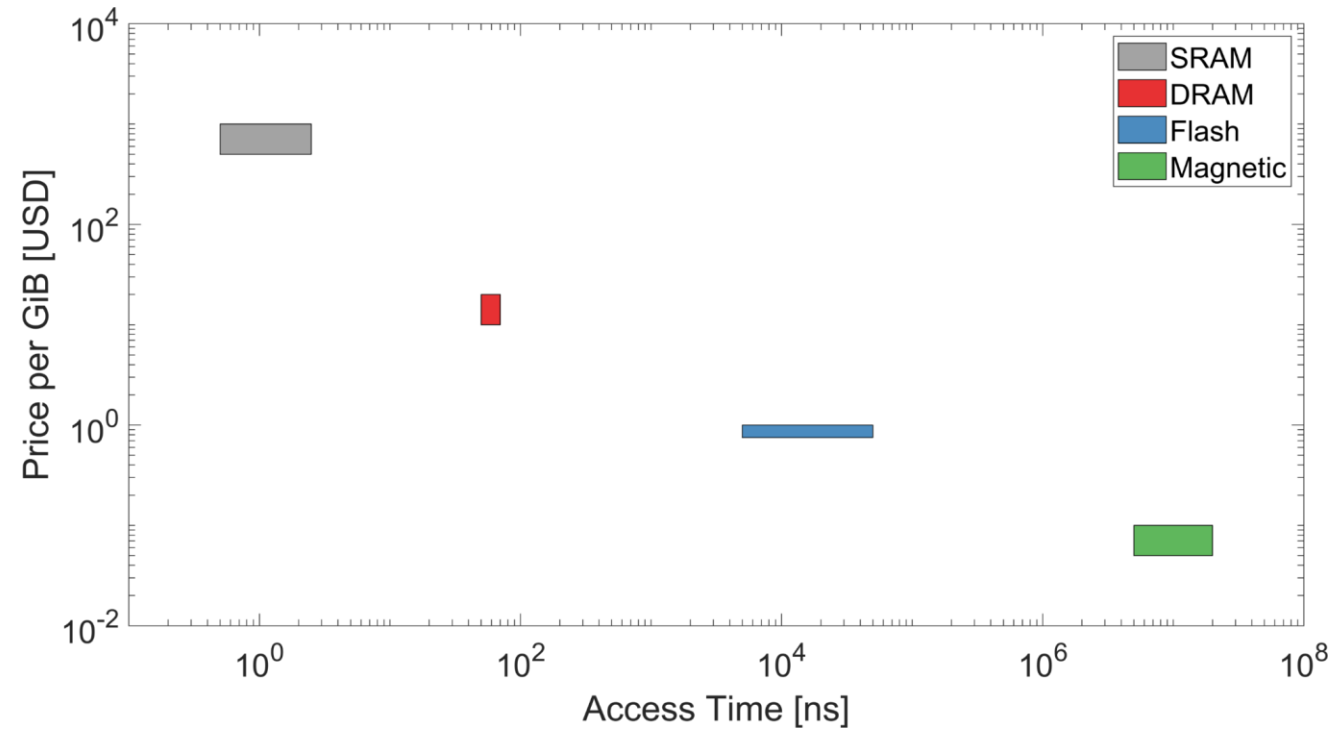


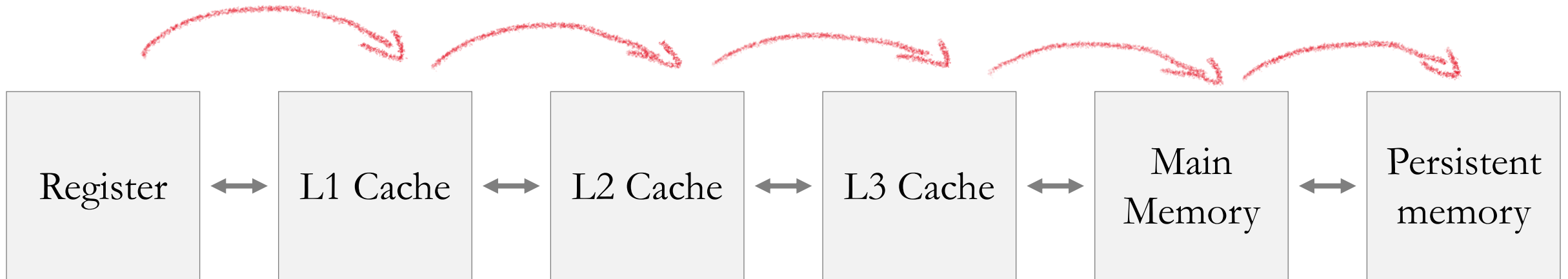
Figure 3. The Memory/Storage Hierarchy in Cost vs. Performance

Source: Objective Analysis



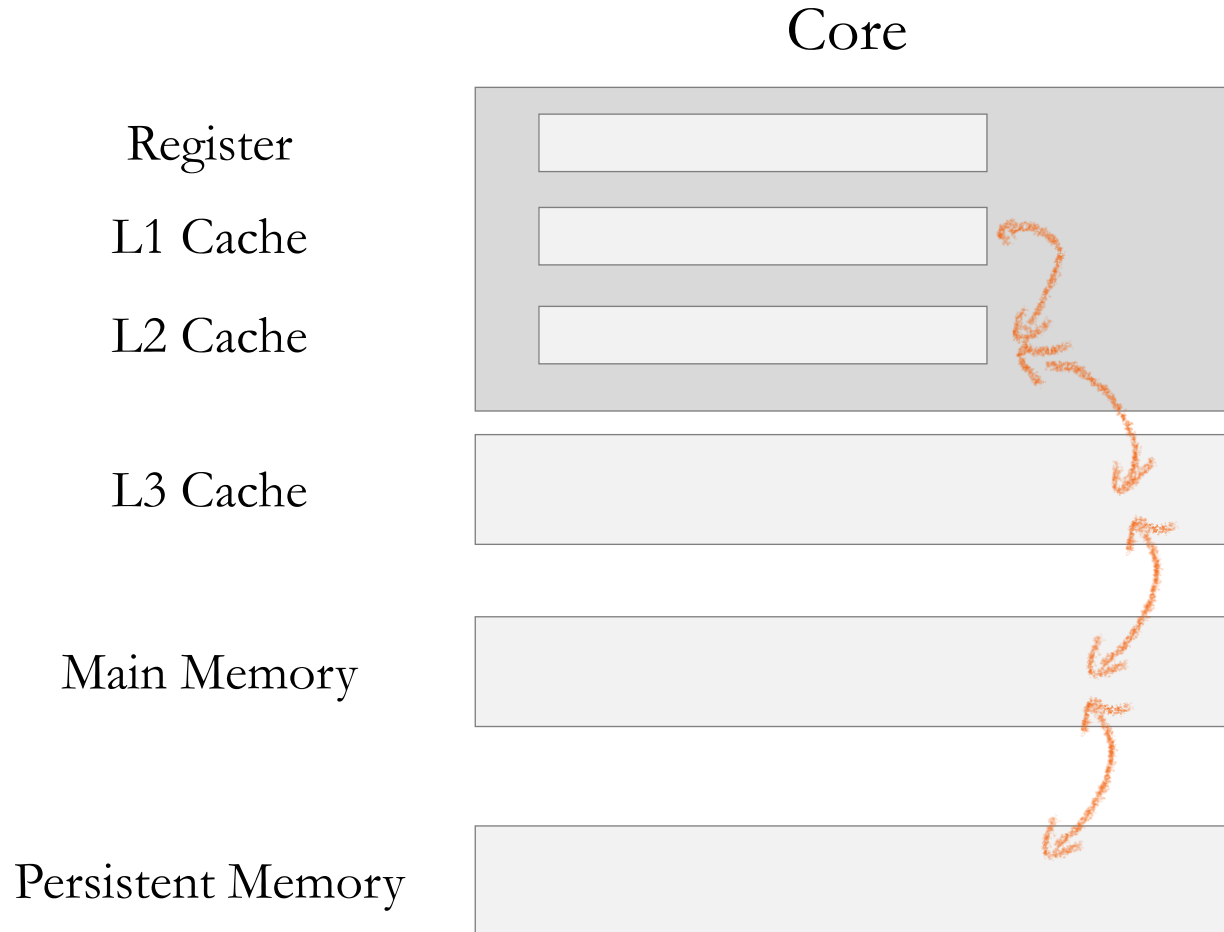
Cache-Prozessfluss

- **L1-Cache:** Kleinster und schnellster On-Chip-Cache; Zuerst nach Registern überprüft.
- **L2-Cache:** Größer und etwas langsamer als L1.
- **L3-Cache:** Noch größer, langsamer und oft zwischen den Kern geteilt.
- **Hauptspeicher:** Viel größer, aber deutlich langsamer als der Cache.
- **Speicher (SSD/HDD):** Der größte und langsamste; Für persistente Daten verwendet.



Die Hierarchie balanciert Geschwindigkeit und Kapazität. Schnellerer Speicher ist kleiner und näher an der CPU, während größerer Speicher langsamer und weiter entfernt ist. Häufig verwendete Daten werden in schnelleren Schichten gespeichert, um die Zugriffszeit zu verkürzen. Falls erforderlich, werden Daten aus der nächstgrößeren Speicherebene angefordert.

Cache Hit und Cache Miss

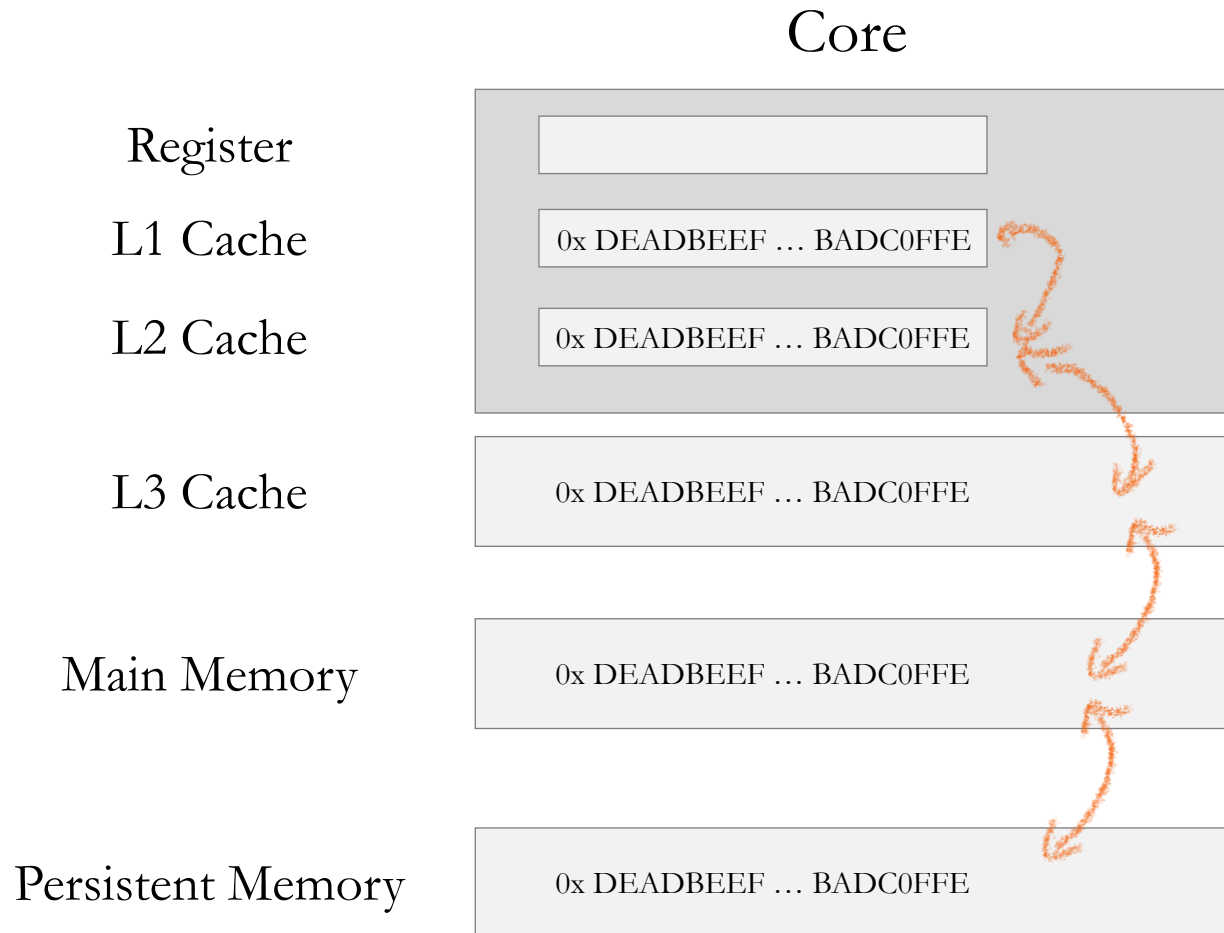


Cache Hit: Ein Cache-Hit tritt auf, wenn der Prozessor bereits im Cache gespeicherte Daten anfordert. Dies führt zu einem schnelleren Abruf, da die benötigten Daten leicht verfügbar sind, was den Bedarf an dem Zugriff auf langsamere Speicherebenen reduziert.

Cache Miss: Ein Cache-Miss tritt auf, wenn angeforderte Daten nicht im Cache vorhanden sind. Daher muss das System die erforderlichen Informationen aus den langsameren Speicherebenen abrufen, was zusätzliche Zeitverzögerungen im Vergleich zu einem Cache-Hit verursacht.

Cache Hit/Miss Ratios: Das Verhältnis beschreibt die Effektivität eines Caches. Das Cache-Hit/Miss-Verhältnis wird berechnet, indem die Anzahl der Cache-Hits/Misses durch die Gesamtzahl der Speicheranfragen geteilt wird, also den Prozentsatz der erfolgreichen/unerfolgreichen Speicheranfragen.

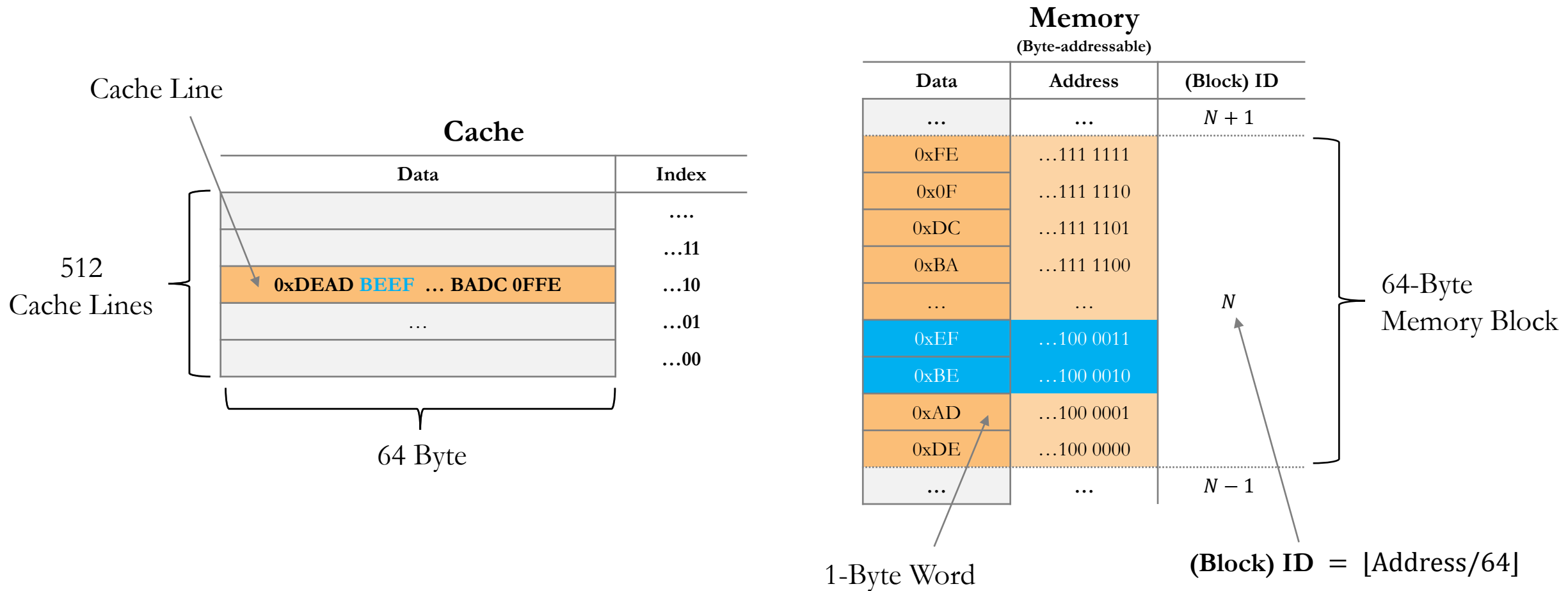
Cache Lines/Blöcke



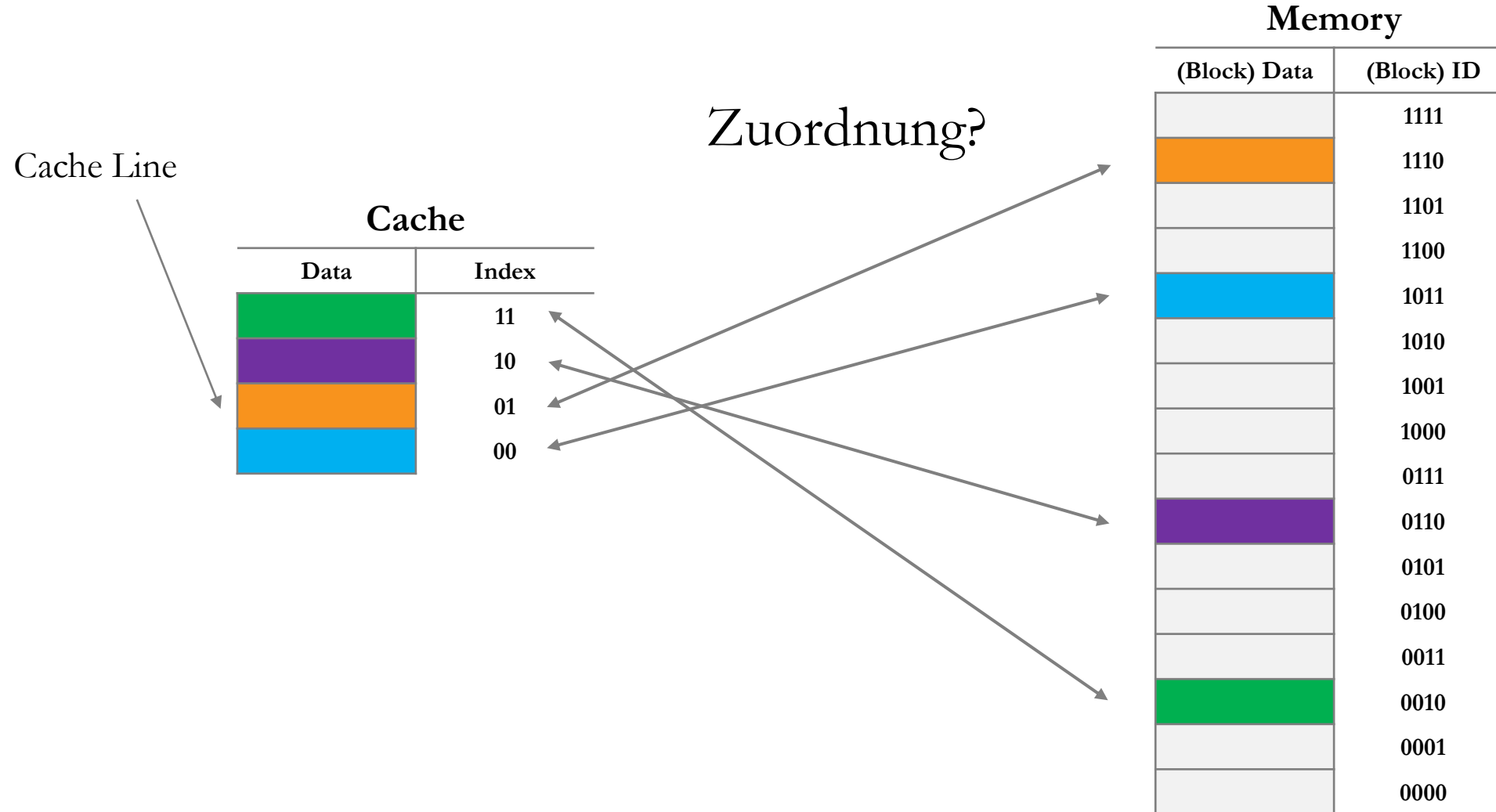
- Der Cache-Speicher wird in Stücke mit fester Größe unterteilt, die **Cache-Lines/Blöcke** genannt werden, die von einem Cache-System verwaltet werden.
- **Cache-Lines** haben eine bestimmte Größe, die als **Cache-Line Größe** bekannt ist. Gängige Größen sind 32, 64 oder 128 Bytes.
- Das Übertragen von Cache-Lines anstelle einer einzelnen Information ermöglicht eine schnellere und effizientere Abrufung von Informationen.
- Bei einer Cache-Line-Größe von 64-Byte holt das Programm, wenn Daten an Adresse X angefordert werden, den gesamten 64-Byte-Chunk mit X ab. Dies nutzt räumliche Lokalität aus und minimiert den Speicherzugriff, wenn nachfolgende Daten (z. B. $X + 10$) benötigt werden, da sie bereits im Cache sind.

Beispiel für Cache-Line

Cache-Speicher mit insgesamt 32KByte = $1024 \cdot 32$ Byte = 32,768 Bytes, der Cache-Lines der Größe 64 Byte verwendet, hat insgesamt $512 = 32,768 \text{ Byte} / 64 \text{ Byte}$ Cache-Lines.

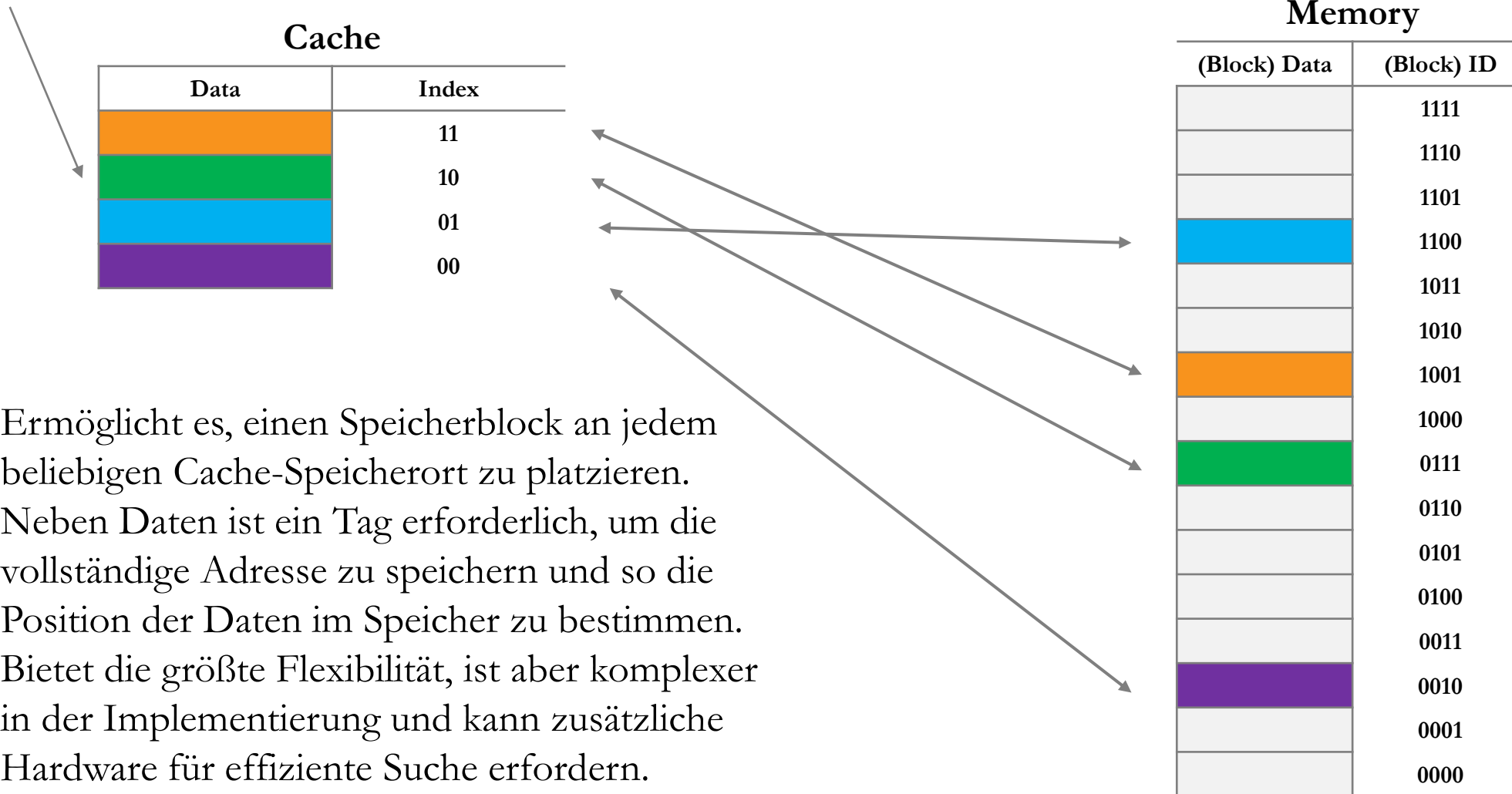


Zuordnungsstrategien für Cache



Voll-Assoziative Zuordnung

Cache Line



Adressierung für voll-assoziative Zuordnung

Die physische Speicheradresse des Wortes im Cache wird durch ein Tag definiert, das den Block im Speicher identifiziert, und einen Offset für die Position innerhalb des Blocks.

Physical Memory Address

Bits for Tag	Offset
--------------	--------

Cache

Data	Tag	Index	Offset
0x0...	...	...	
0x0D 0x15 0xEA 0x5E	...01	...	10
0x0...	...	...	

0xEA

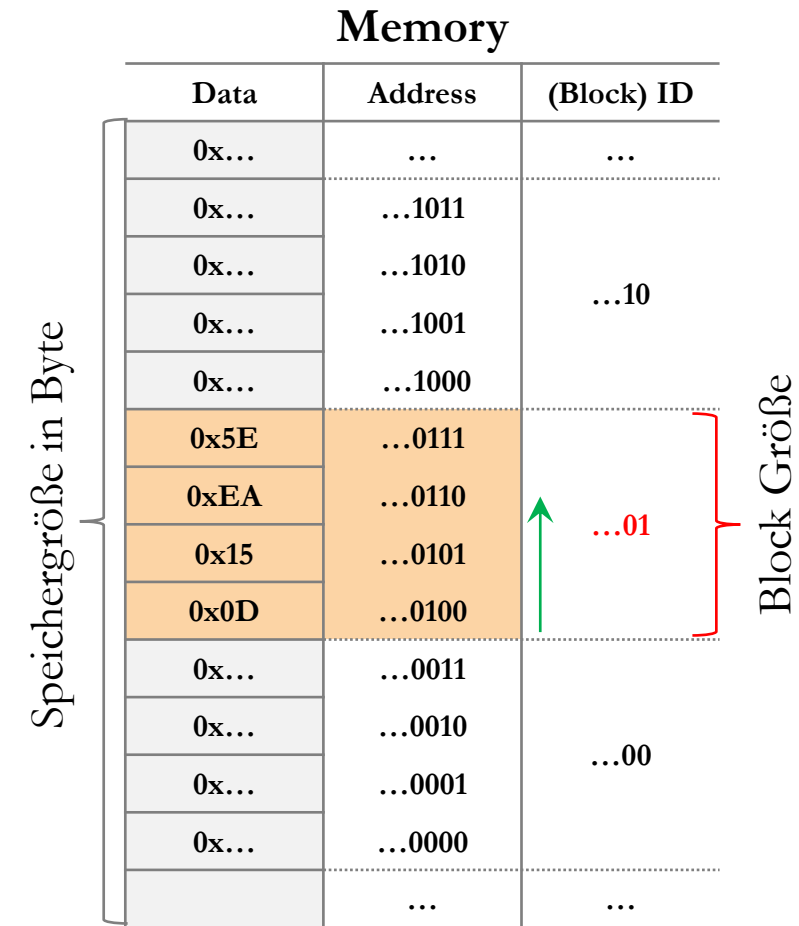
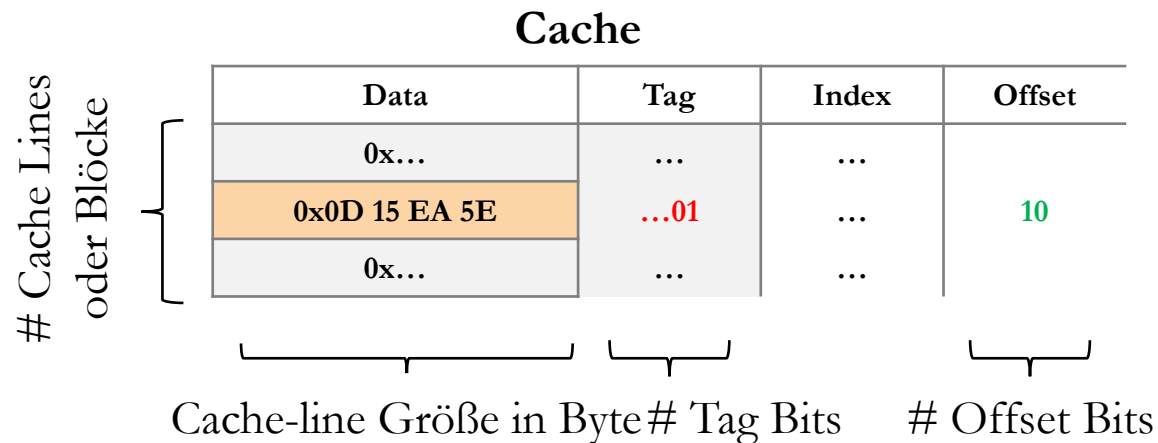
Memory

Data	Address	(Block) ID
0x...	...	...
0x...	...1011	
0x...	...1010	
0x...	...1001	...10
0x...	...1000	
0x5E	...0111	
0xEA	...0110	...01
0x15	...0101	
0x0D	...0100	
0x...	...0011	
0x...	...0010	
0x...	...0001	...00
0x...	...0000	
	...	...

Adressierung für voll-assoziative Zuordnung

Die Zerlegung der physischen Speicheradresse hängt von der Größe von Speicher-, Cache und Cache-Lines(/Blöcken) ab.

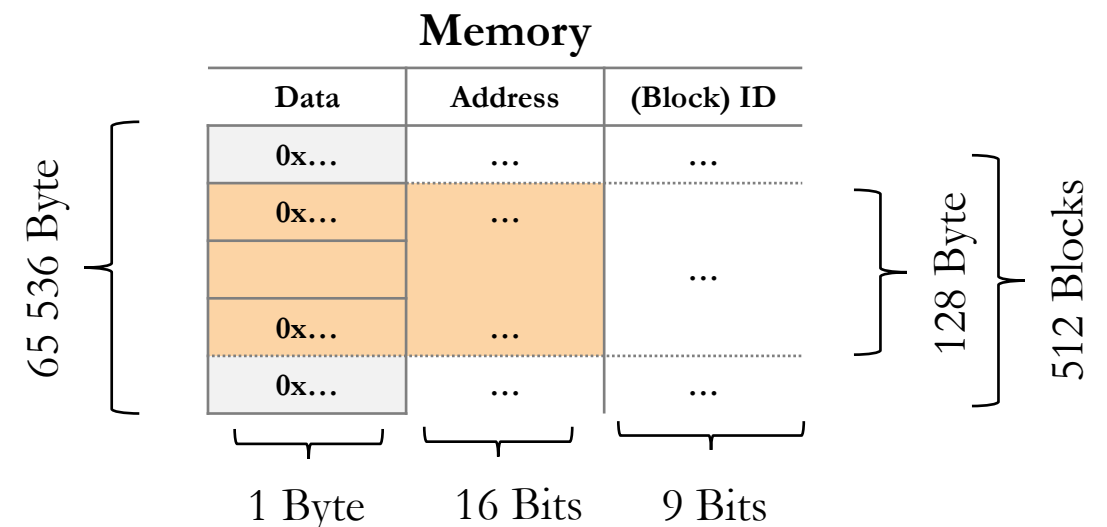
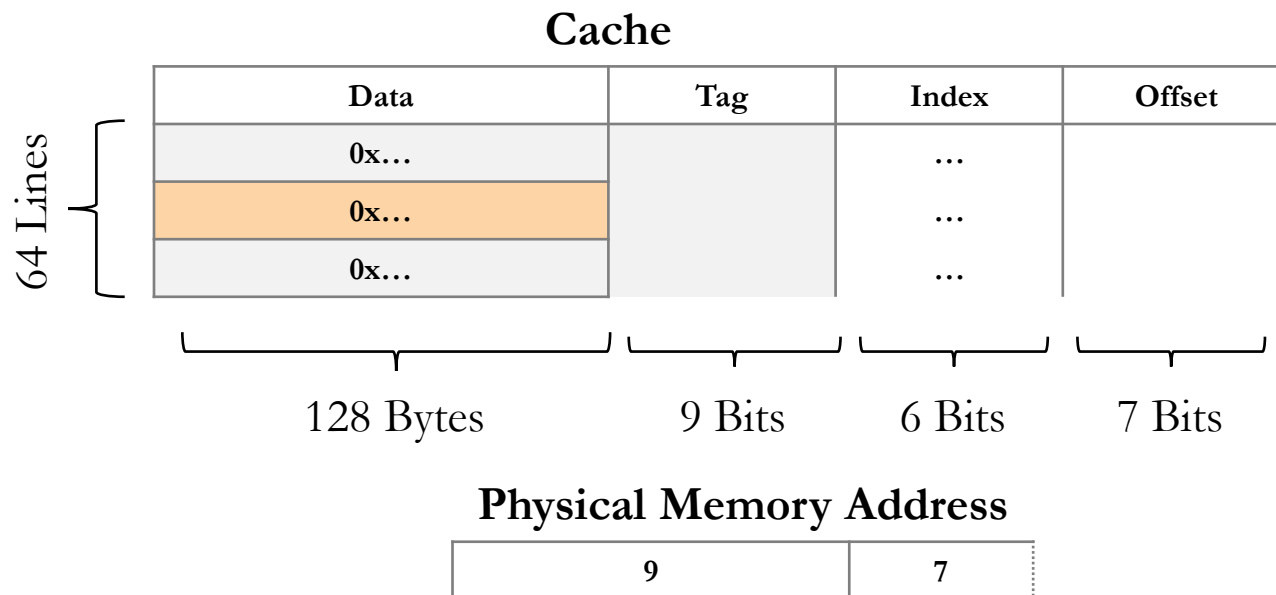
Anzahl der Bits für die Speicheradresse	$\log_2(\text{Speichergröße in Byte})$
Anzahl der Bits für Offset innerhalb des Blocks	$\log_2(\text{Cache-Line-Größe in Byte})$
Anzahl der Tag-Bits (zur Blockidentifikation)	Anzahl der Bits für physische Adresse – Anzahl der Bits für den Offset innerhalb des Blocks
Anzahl der Cache-Zeilen oder -blöcke	Cache-Größe / Blockgröße



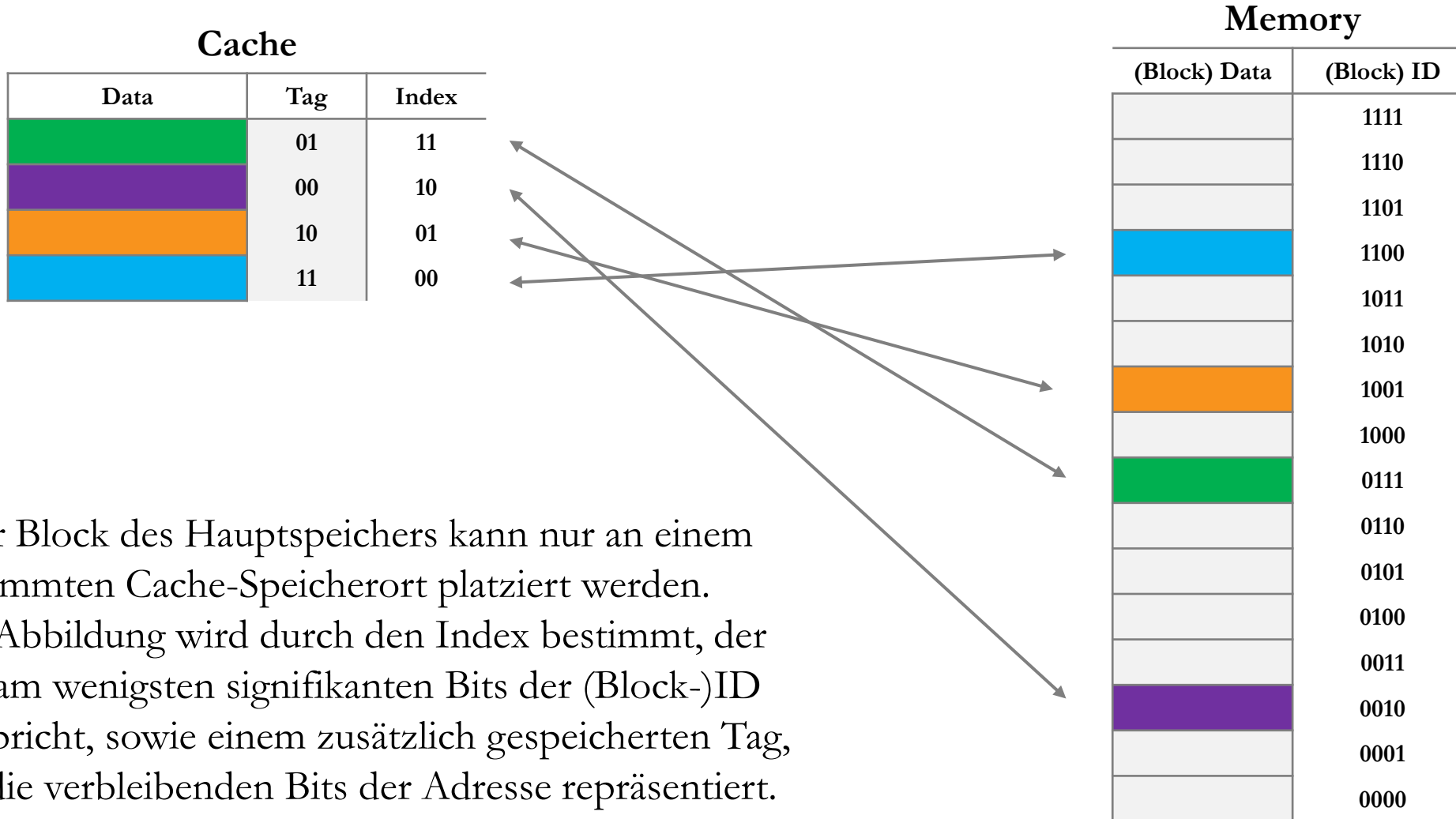
Adressierung für voll assoziative Zuordnung

Beispiel: Cache mit 8 KB Größe, Cache-Zeilengröße = 128 Byte, Größe des Hauptspeichers = 64 KB

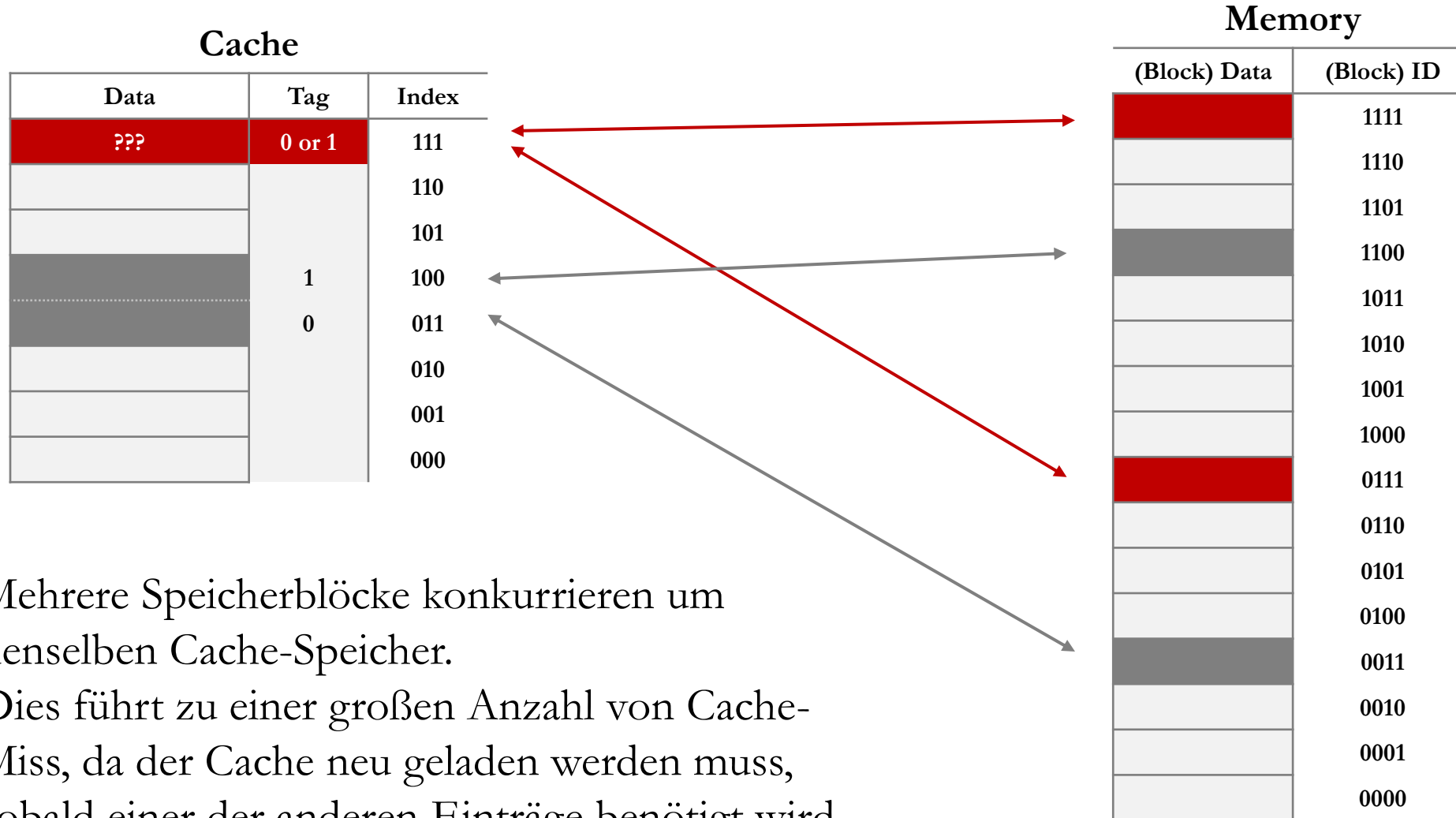
Anzahl der Bits für Speicheradresse	$\log_2(\text{Memory size in Byte})$	$\log_2(65.536) = 16$
Anzahl der Bits für Offset innerhalb des Blocks	$\log_2(\text{Cache-line size in Byte})$	$\log_2(128) = 7$
Anzahl der Tag-Bits	Anzahl der Bits für Speicheradresse – Anzahl der Bits für den Offset innerhalb des Blocks	$16 - 7 = 9$
Anzahl der Cache-Zeilen oder -blöcke	Cache Größe (in Byte) / Block Größe (in Byte)	$8192 / 128 = 64$
Anzahl der Speicherblöcke	Memory Größe (in Byte) / Block Größe (in Byte)	$65\,536 / 128 = 512$



Direkte Zuordnung

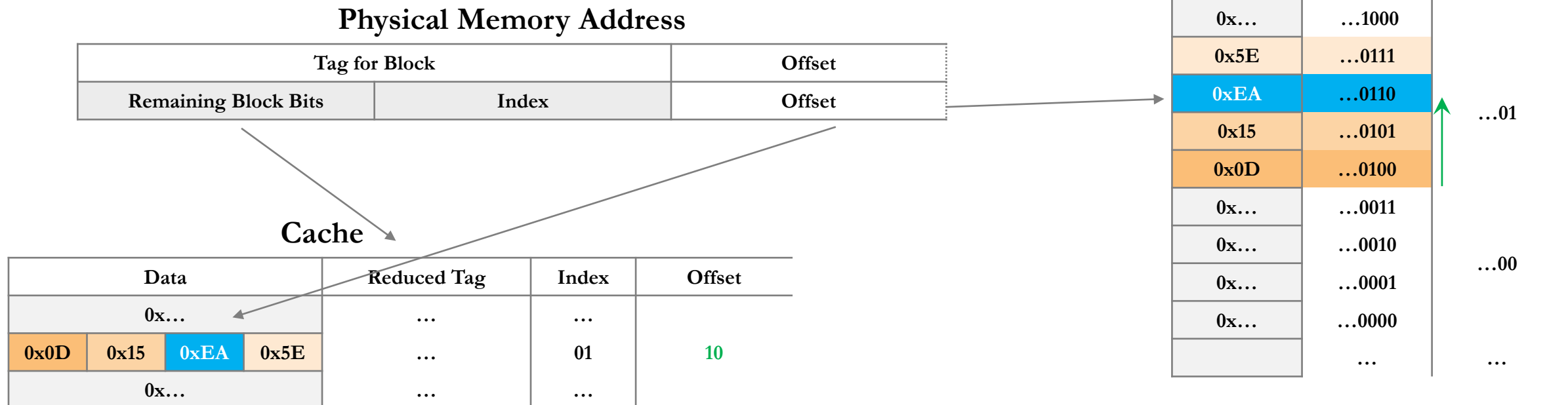


Konflikte bei direkter Zuordnung



Adressierung für direkte Zuordnung

Ähnlich wie bei der assoziativen Zuordnung erfordert diese Zuordnung einen Offset für die Position innerhalb des Blocks. Im Gegensatz zur assoziativen Zuordnung müssen nicht alle Bits für das Tag (zur Identifikation des Blocks) gespeichert werden, da die letzten Bits bereits durch den Index der Cache-Zeile gegeben sind.



Set-Assoziative Zuordnung

Set/Groups werden auch Gruppen/Mengen genannt.

Cache with 4 Groups

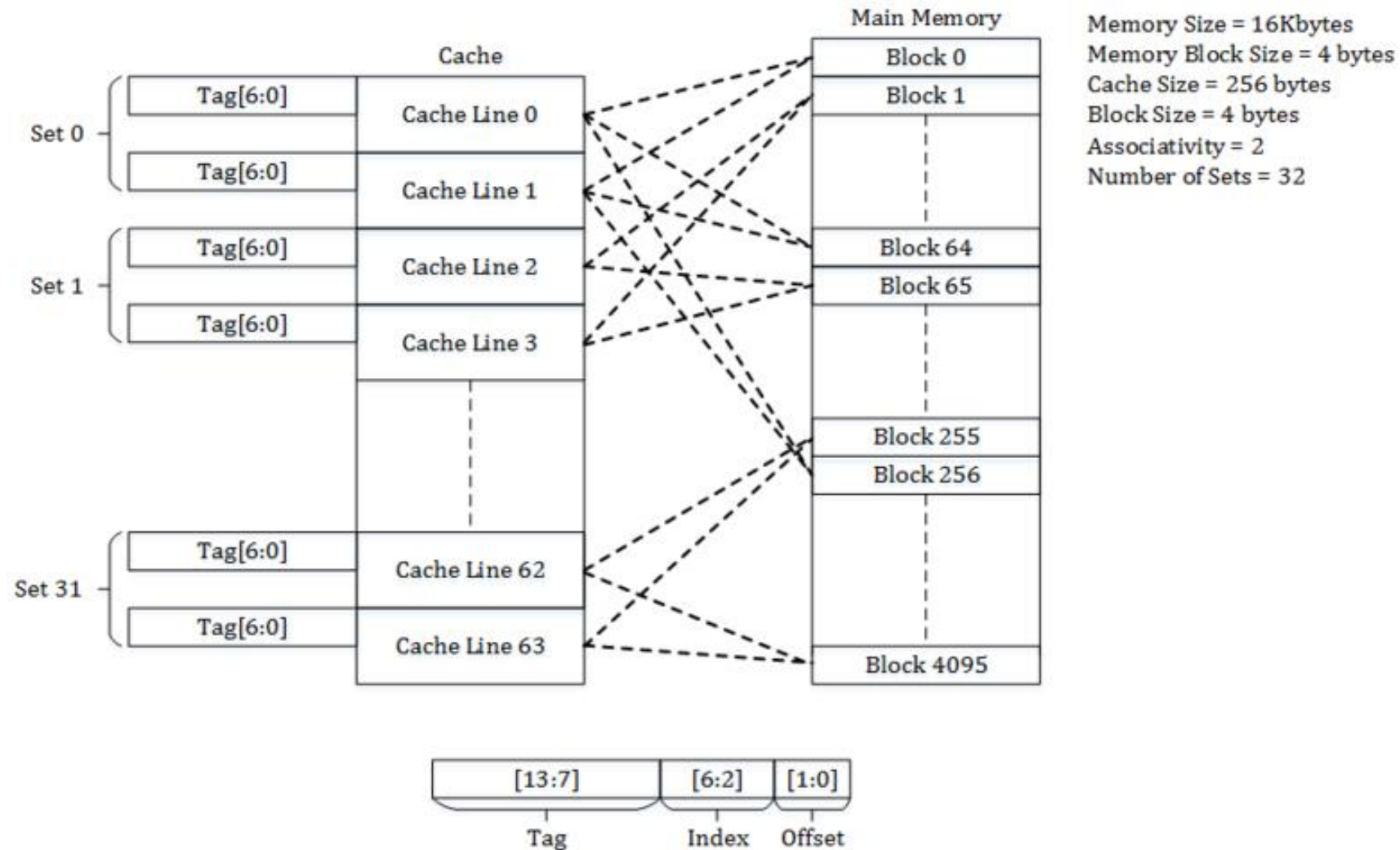
Data	Tag	Index	Group
	01 or 11	11	3
	01 or 11		
		10	2
		01	1
	00		
	11	00	0

Memory

(Block) Data	(Block) ID
	1111
	1110
	1101
	1100
	1011
	1010
	1001
	1000
	0111
	0110
	0101
	0100
	0011
	0010
	0001
	0000

- teilt den Cache in mehrere Cache-Gruppen auf, und jede Gruppe enthält mehrere Cache-Linien.
- Ein Speicherblock kann in jede der Cache-Zeilen innerhalb einer bestimmten Gruppe platziert werden.
- Bietet ein Gleichgewicht zwischen Einfachheit (im Vergleich zur direkten Zuordnung) und Flexibilität (im Vergleich zur vollständig assoziativen Zuordnung).

2-way Set-Assoziatives Beispiel



L-way Set-Assoziative Zuordnung

L-way Set assoziative Zuordnung ist ein Kompromiss zwischen der Flexibilität der voll assoziativen Zuordnung, bei der ein Block überall im Cache platziert werden kann, und der Einfachheit der direkten Zuordnung, bei der jeder Block einen festen Platz im Cache hat. Es unterteilt den Cache in Mengen mit jeweils L Cache-Lines. Blöcke aus dem Hauptspeicher werden direkt auf die entsprechende Menge abgebildet. Innerhalb der Menge ist die Abbildung assoziativ.

1-way Set-Associative
8 Sets with 1 Block each

Data	Tag	Index	Set
		111	7
		110	6
		101	5
		100	4
		011	3
		010	2
		001	1
		000	0

2-way Set-Associative
4 Sets with 2 Blocks each

Data	Tag	Index	Set
		11	3
		10	2
		01	1
		00	0

4-way Set-Associative
2 Sets with 4 Blocks each

Data	Tag	Index	Set
		1	1
		0	0

8-way Set-Associative
1 Set with 8 Blocks each

Data	Tag	Index	Set
		NA	0

Direkte Zuordnung

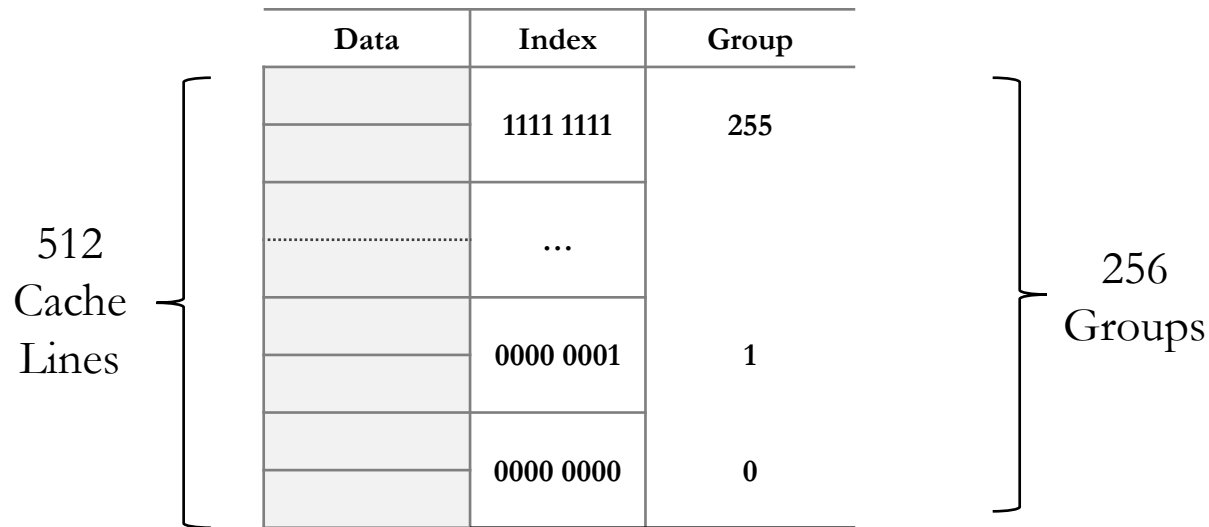


Voll Assoziative Zuordnung

L1 Data-Cache for Cortex-A72

L1 Instruction-Cache Kapazität	L1 Instruction-Cache Aufbau	L1 Data-Cache Kapazität	L1 Data-Cache Aufbau	L2 Cache Kapazität	L2 Cache Aufbau
48 KB (per core)	3-way set associative, 64-byte cache line	32 KB (per core)	2-way set associative, 64-byte cache line	1 MB (shared)	shared, 16-way set associative, 64-byte cache line

L1 Data-Cache

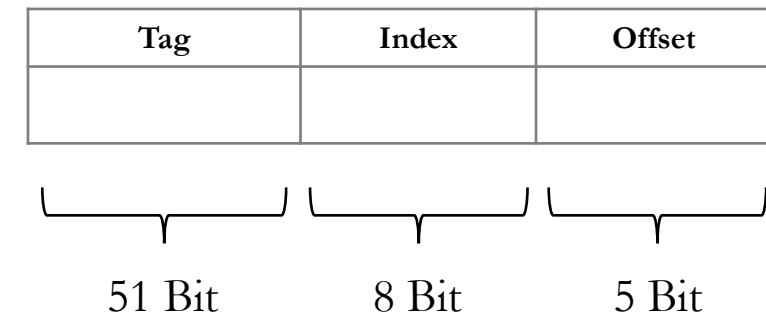


64 Byte per Cache Line

Cache Lines = 32 KB / 64 Byte = 512

Groups = # Cache Lines / L = 512 / 2 = 256

64-Bit Adresse



Richtlinien für Cache-Austausch

Eine Richtlinie für Cache-Austausch definiert, welche Daten in einem Cache für den Austausch ausgewählt werden, wenn neue Daten gespeichert werden müssen. Gängige Strategien sind:

- **Zufälliger Austausch:** Wählt Cache-Linie zufällig bei einem Cache-Fehlschlag aus. Obwohl es einfach zu implementieren ist, nutzt dies den Cache möglicherweise nicht effizient aus.
- **First-In-First-Out (FIFO):** Ersetzt die Cache-Linie, die am längsten im Cache war, und verwendet zur Einfachheit einen kreisförmigen Puffer. Allerdings wird die Cache-Nutzung nicht immer effektiv optimiert.
- **Least-Recently-Used (LRU):** Ersetzt die zuletzt abgerufene Cache-Zeile und erfordert eine Nachverfolgung der Nutzungshistorie. Das kann rechenintensiv sein.
- **Least-Frequently-Used (LFU):** Priorisiert basierend auf Nutzungshäufigkeit und entfernt zuerst weniger häufig aufgerufene Elemente, ähnlich wie bei LRU, aber mit der Verfolgung der Zugriffshäufigkeit statt der Aktualität.
- ...

Cache mit 4 Gruppen

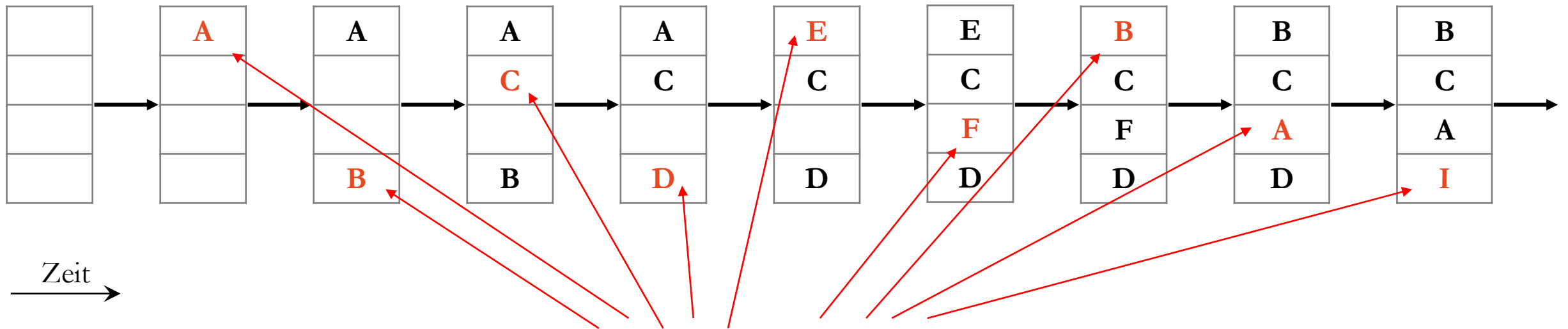
Data	Tag	Index	Group
	01 or 11	11	3
	01 or 11		
		10	2
		01	1
		00	0

Memory

(Block) Data	(Block) ID
	1111
	1110
	1101
	1100
	1011
	1010
	1001
	1000
	0111
	0110
	0101
	0100
	0011
	0010
	0001
	0000

Zufälliger Austausch

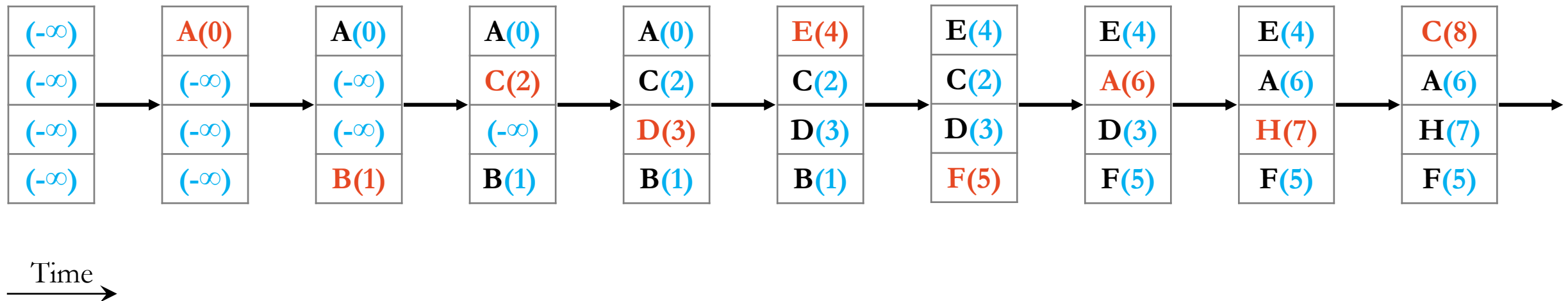
Wählt zufällig eine Cache-Linie aus, welche ersetzt werden soll, um neue Daten bei einem Cache-Miss zu speichern.



Cache Miss – Benötigte Daten
werden aus dem Speicher in die
Cache-Zeile geladen

First-In-First-Out (FIFO)

Ersetzt die Cache-Zeile, die am längsten im Cache war. Neben den Daten erfordert dies auch einen Zähler, der speichert, wann eine Cache-Zeile geladen wurde.

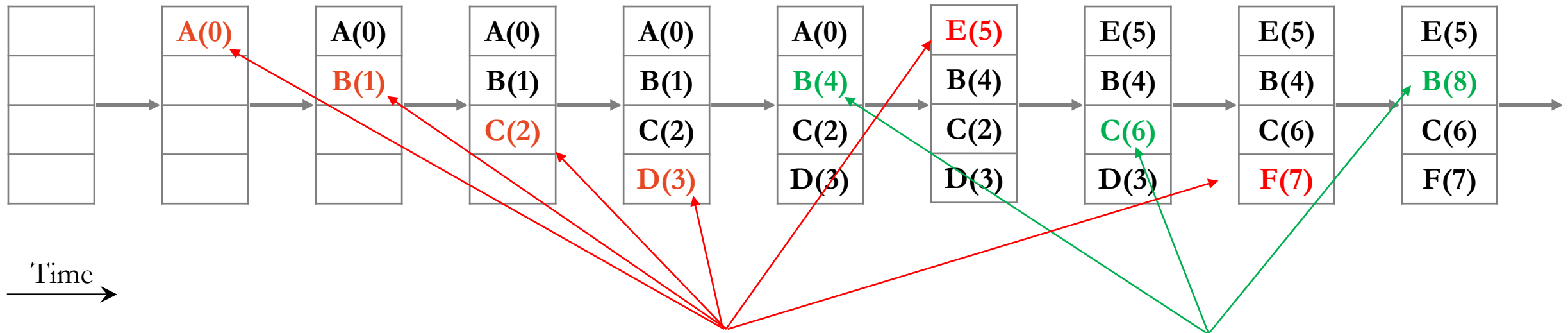


$X(\bullet)$ = Zählt, wenn die Daten X in die Cache-Zeile geladen wurden

Cache Miss – Benötigte Daten werden aus dem Speicher in die Cache-Zeile geladen

Least Recently Used (LRU)

Ersetzt die zuletzt abgerufene Cache-Linie und erfordert einen Zähler, der die Nutzungshistorie verfolgt, also wann die letzte Cache-Linie abgerufen wurde.



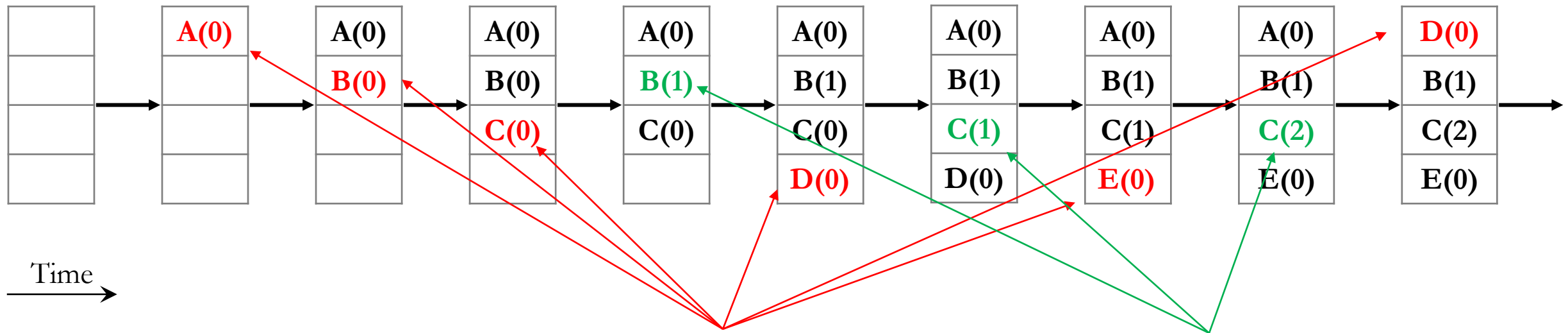
$X(\bullet)$ = Zählt, wann die letzte
Verwendung von Daten X in der Cache-
Linie war

Cache Miss – Benötigte Daten
werden aus dem Speicher in die
Cache-Zeile geladen

Cache-Treffer – Daten wurden in
der Cache-Linie gefunden und
erfolgreich abgerufen

Least Frequently Used (LFU)

Ersetzt die am seltensten genutzte Cache-Linie und erfordert einen Zähler, der die Nutzungshistorie verfolgt – also wie oft eine Cache-Linie erfolgreich verwendet wurde.

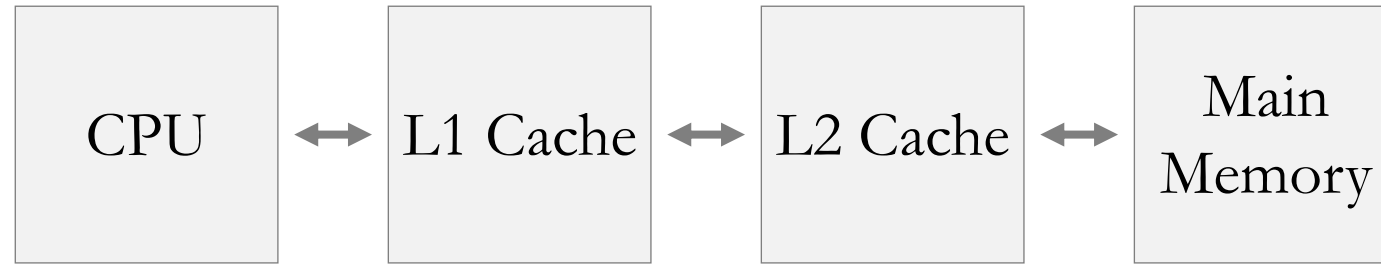


$X(\bullet)$ = Zählt, wie oft die Daten X
in der Cache-Linie verwendet
wurden

Cache Miss – Benötigte Daten
werden aus dem Speicher in die
Cache-Zeile geladen

Cache-Treffer – Daten wurden in
der Cache-Linie gefunden und
erfolgreich abgerufen

Speicherschichten für Raspberry PI 4



Raspberry Pi 4 Model B Rev 1.4

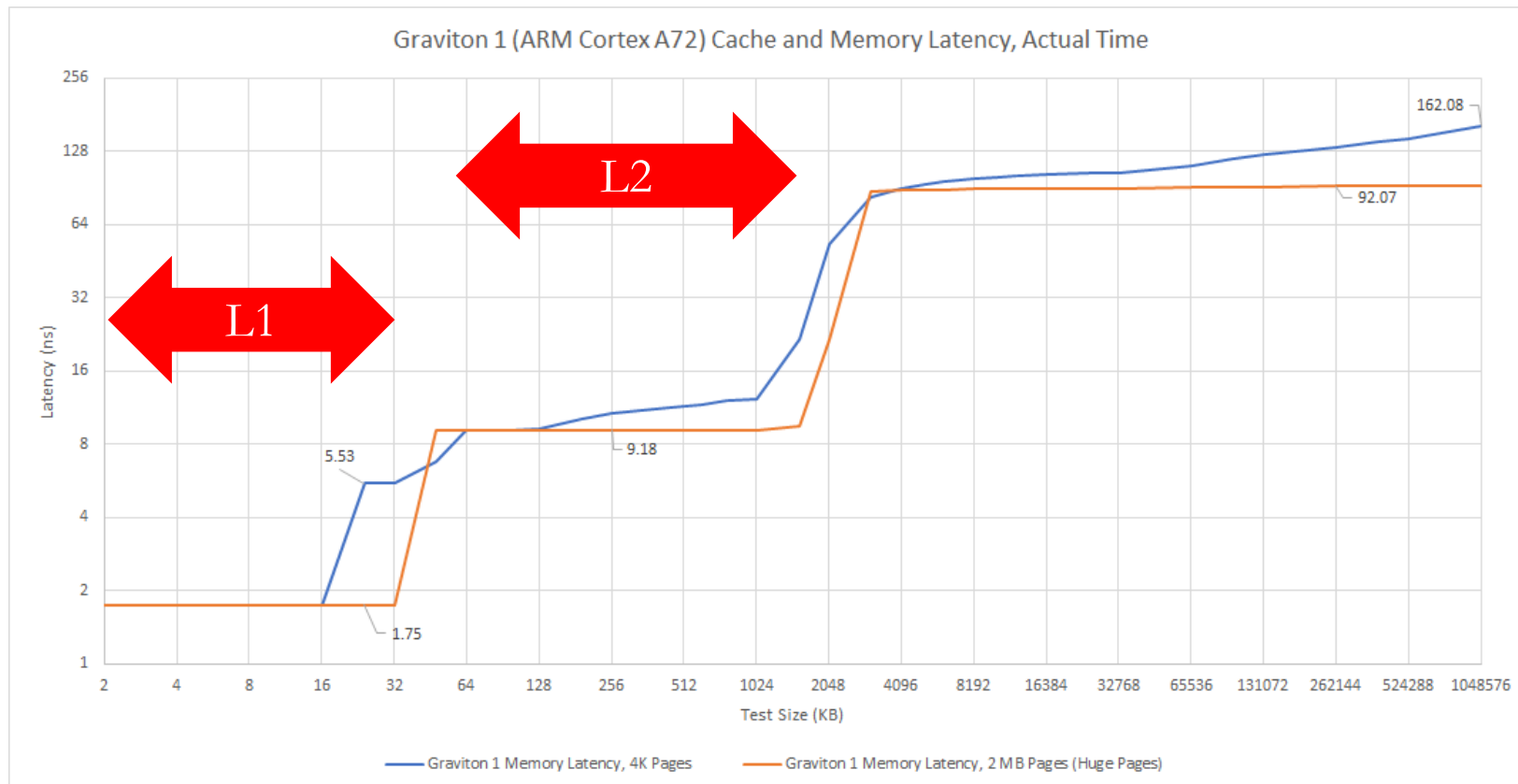
- CPU: Broadcom BCM2711, Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1500 MHz
- L1D (Data) cache:
 $128 \text{ KB} = 4 \times 32 \text{ KB}$
- L1I (Instruction) cache:
 $192 \text{ KB} = 4 \times 48 \text{ KB}$
- L2 cache: 1 MB (Shared between cores)
- Main Memory: 8 GB LPDDR4-3200 SDRAM

```
cu62xax@rspi08:~$ lscpu
Architecture: aarch64
CPU op-mode(s): 32-bit, 64-bit
Byte Order: Little Endian
CPU(s): 4
On-line CPU(s) list: 0-3
Vendor ID: ARM
Model name: Cortex-A72
Model: 3
Thread(s) per core: 1
Core(s) per cluster: 4
Socket(s): -
Cluster(s): 1
Stepping: r0p3
CPU max MHz: 1500,0000
CPU min MHz: 600,0000
BogoMIPS: 108.00
Flags: fp asimd evtstrm crc32 cpuid
Caches (sum of all):
L1d: 128 KiB (4 instances)
L1i: 192 KiB (4 instances)
L2: 1 MiB (1 instance)
Vulnerabilities:
Gather data sampling: Not affected
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Retbleed: Not affected
Spec rstack overflow: Not affected
Spec store bypass: Vulnerable
Spectre v1: Mitigation; __user pointer sanitization
Spectre v2: Vulnerable
Srbds: Not affected
Tsx async abort: Not affected
```

```
rspi08.inf-ra.uni-jena.de
description: Computer
product: Raspberry Pi 4 Model B Rev 1.4
serial: 10000000d53012b7
width: 64 bits
capabilities: smp cp15_barrier setend swp tagged_addr_disabled
*-core
description: Motherboard
physical id: 0
*-cpu:0
description: CPU
product: cpu
physical id: 1
bus info: cpu@0
size: 1500MHz
capacity: 1500MHz
capabilities: fp asimd evtstrm crc32 cpuid cpufreq
*-cache
description: L1 Cache
physical id: 0
size: 32KiB
*-cpu:1
description: CPU
product: cpu
physical id: 2
```

Speicher-Latenz

Speicher-Latenz bezieht sich auf die Dauer zwischen dem Moment, in dem eine Speicherleseinstruktion initiiert wird, und dem Moment, in dem die entsprechenden Daten in einem Speicherregister zugänglich sind.



Source: <https://chipsandcheese.com/2023/11/10/arms-cortex-a72-aarch64-for-the-masses/>

Graviton

General information

Launched November 26, 2018; 5 years ago

Performance

Max. CPU clock rate 2.3 GHz

Cache

L1 cache 80 KB per core (48 instructions + 32 data)

L2 cache 8 MB total

Architecture and classification

Technology node 16 nm

Instruction set AArch64

Instructions AArch64

Extensions Neon, crc, crypto

Physical specifications

Cores 16x Cortex A72

History

Successor(s) Graviton2

Support status

Supported

Source: https://en.wikipedia.org/wiki/AWS_Graviton

Motivation

Problem: $z \leftarrow z + x \cdot y$ mit Matrizen $z, x, y \in \mathbb{R}^{n \times n}$

Algorithmus:

```
for i=1,2,...n
  for j=1,2,...n
    for k=1,2,...n
       $z(i,j) \leftarrow z(i,j) + x(i,k) \cdot y(k,j)$ 
```

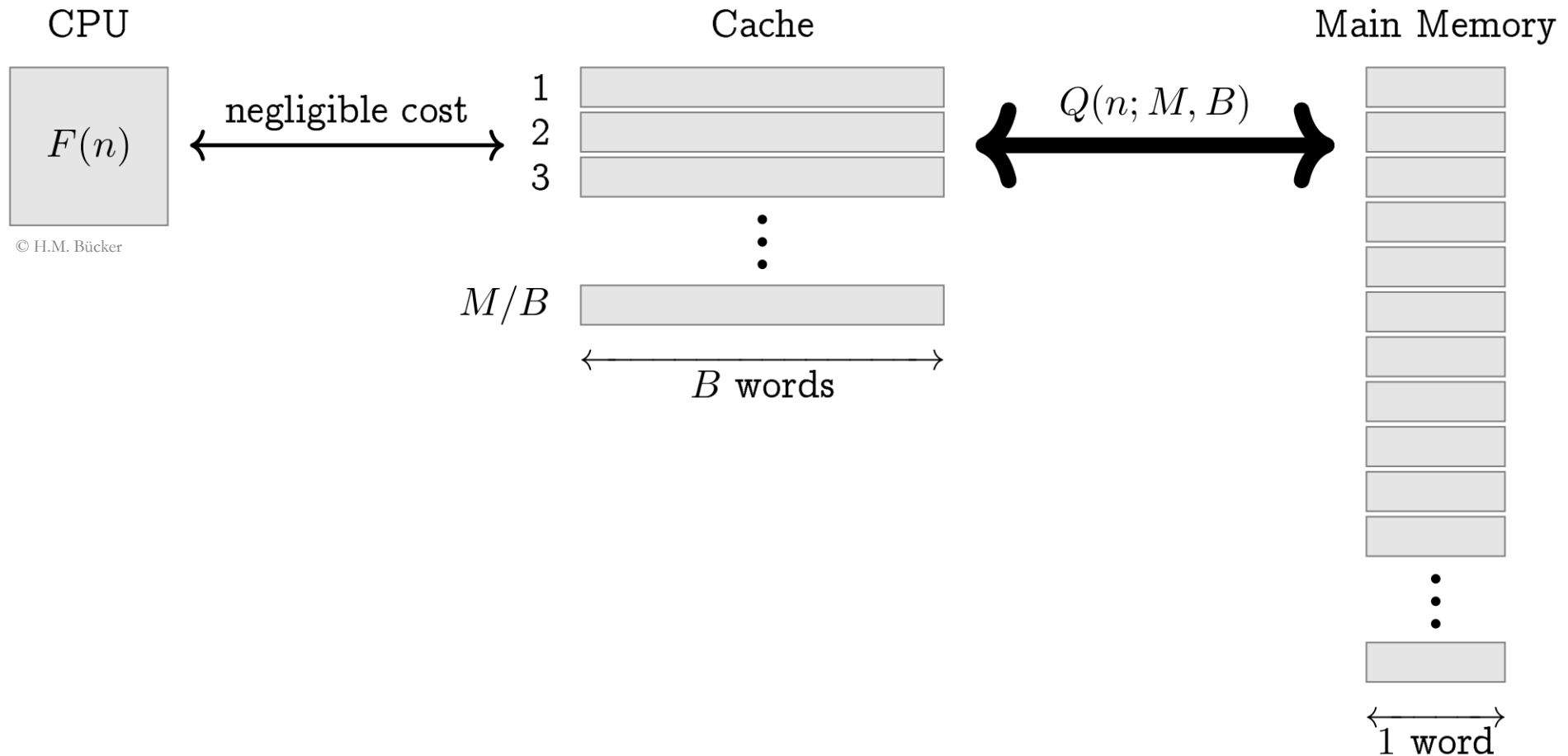
Nur Operationen zählen und Kosten für Speicherzugriff/Datenübertragung ignorieren ($\alpha \gg \beta$):

$$\Rightarrow T = W(n) = \Theta(n^3)$$

Ein realistischeres Modell sollte jedoch auch Speicherzugriff mit Caching-Effekten berücksichtigen.

Ideales Cache Modell

Einfaches Modell für eine 2-stufige Speicherhierarchie mit
schnellem Cache und langsamem Hauptspeicher.



Ideales Cache Modell Annahme

- Zweistufige Speicherhierarchie, bestehend aus Cache und Hauptspeicher.
- Der Hauptspeicher ist beliebig groß.
- Die CPU verarbeitet nur Daten im Cache.
- Der Cache ist begrenzt.
- **Cache Size:** $M \in \mathbb{N}$ Words (Wort ist kleinste Dateneinheit, Wortgröße ist kleine Konstante).
- Der Cache ist in **Cache-Lines** unterteilt, die aus einer Folge von Wörtern bestehen.
- Cache-Lines haben die gleiche **Größe:** $B \in \mathbb{N}$ (Anzahl der Wörter pro Cache-Line).
- Annahme **Schlanker Cache:** #Cache-Lines M/B ist viel größer als #Wörter pro Cache-Zeile

$$M = \Omega(B^2)$$

- Daten werden zwischen Cache und Hauptspeicher in Form von Cache-Lines übertragen.

Ideales Cache Modell Annahme

Wenn Prozessor auf Daten/Wort zugreifen, gibt es zwei mögliche Situationen:

- **Cache Hit:** Wort ist bereits Teil eines Blocks, der im Cache vorhanden ist.
- **Cache Miss:** Wort ist nicht im Cache vorhanden und muss daher vom Hauptspeicher in den Cache geladen werden.

Idealer Cache Austausch bei einem Cache Miss:

- Neue Daten können überall im Cache gespeichert werden (voll assoziativ).
- Ersetze Daten, welche in Zukunft am längsten nicht verwendet werden.

Zwei Bewertungskriterien für einen Algorithmus mit Problemgröße $n \in \mathbb{N}$:

- **Work:** Zähle die Anzahl der Operationen $W(n)$.
- **Cache Complexity:** Zähle die Anzahl der Cache-Misses $Q(n; M; B)$.

Strided-Zugriff mit Stride-Größe 1

```
for i = 0,...,N-1  
    s = s + a[i];
```

- Variable **s**: hohe temporale Lokalität
- Variable **a**: hohe räumliche Lokalität
- Zugriffsmuster: $a[0], a[1], a[2], \dots, a[N-1]$
- Zugriffsreihenfolge: $1 \quad 2 \quad 3 \quad \dots \quad N$
- Speicheradresse: $0 \quad 1*T \quad 2*T \quad \dots \quad (N-1)*T$

Allgemeines Striding mit Stride-k = auf jedes k-te Element wird zugegriffen.

Zufälliger Zugriff

```
for i = 0,...,N-1  
    s = s + a[Index(i)];
```

i	0	1	2	3	4	5
Index	17	312	101	505	1	250

- Zugriffsmuster: $a[17], a[312], a[101], \dots$
- Zugriffsreihenfolge: 1 2 3 ...
- Speicheradresse: $17 \cdot T$ $312 \cdot T$ $101 \cdot T$...

Geringe räumliche und zeitliche Lokalität bezüglich der Variablen a .